Benchmarking

This document defines how BAYMAX evaluation scenarios are written, scored, and reported.

Final Local Development Results (2026-10-17)

The owner closed out the stepping-stone project after these saved evaluations:

Run	Scenarios	Task successes	Macro-F1	Invalid outputs
Unweighted name-first reference, 1k / 2 epochs	150	57	0.612	1
Grounding v1, 1k / 2 epochs	150	53	0.591	0
Coverage v2, 2k / 2 epochs	150	64	0.615	2
Coverage v2, 2k / 4 epochs	150	73	0.681	0

The four-epoch raw result records ACT routing 62/81, CLARIFY 17/40 and REFUSE 27/29. All four runs use compact JSON and greedy generation on the combined v1/v2 suite through fake adapters. The expanded artifact has 1,952 usable training examples and 195 validation examples at context 1,024; the raw caps are 2,000 and 200.

These are single-seed, repeatedly inspected development measurements, not untouched-test estimates or statistically established gains. Do not compare the 150-case headline directly with a 50-case hosted baseline. Keyword-based question and reason scores remain distinct from routing macro-F1. Latency in the final run averaged 5,652 ms, versus 2,758 ms for its two-epoch counterpart, without a controlled hardware timing study. The local API cost field is not total operating cost. See the complete experiment report for settings, source links and the decision matrix. No evaluation target or historical result was changed for closeout.

Phase 1 starts with scripted scenarios and fake tool adapters so the evaluation harness is reproducible before any real APIs are introduced. The goal is to measure agent behaviour in a controlled environment before connecting BAYMAX to live Notion, Google Calendar, Gmail, or clipboard integrations.

Scenario Difficulty Labels

Each scenario must include one difficulty label:

- `explicit`: The user gives exact date, time, or task details directly.
- `implicit`: The user uses relative or natural phrasing that must be resolved using `current_time`, such as "tomorrow", "next Friday", or "in two hours".
- `contextual`: The agent must inspect `initial_state` to identify the correct entity or action.

- `ambiguous`: The agent should ask a clarification question instead of taking action.
- `multi_step`: The agent must call more than one tool to complete the task.

Canonical Decision Policy

Training examples and evaluation scenarios must apply the same decision policy. Decisions are evaluated in this order: `REFUSE`, then `CLARIFY`, then `ACT`. An earlier matching decision takes precedence over later ones.

Decision	Required conditions
`REFUSE`	The requested capability is unavailable, unsupported by the available tool schemas, out of scope, or unsafe. A related tool does not make the operation supported, and the agent must not substitute a different operation.
`CLARIFY`	The action is supported and safe, but required information is missing, ambiguous, conflicting, invalid, or cannot be resolved to one entity from `initial_state`. Clarification also applies when the confirmation policy below requires confirmation.
`ACT`	The action is supported and safe, every required argument can be resolved from the request, `current_time`, or `initial_state`, and no confirmation is required.

Examples of unsupported capability include requesting email search when only draft and send tools are available, or requesting deletion when the available tool can only update an entity. Confirmation does not make an unsupported or unsafe action valid; those requests remain refusals.

Optional Task Deadlines

Notion task creation permits an omitted due date. Bare urgency such as "create a task to revise a topic soon" permits ACT with an undated task; preserve the urgency in the task text rather than inventing a deadline. An explicit deadline request such as "due soon" or "due on the deadline we agreed" requires CLARIFY when the requested date cannot be grounded.

This distinction was confirmed on 2026-10-14 and is represented in synthetic generator version 15. Historical datasets, scenario targets and result scores are not rewritten or reinterpreted by that generator change.

Confirmation Policy

Confirmation is required before:

- a destructive or irreversible operation that an available tool actually supports, such as deleting or cancelling an existing entity; or
- an externally visible communication that states a consequential commitment or status change, such as withdrawing from a course.

Confirmation is not required solely because an action mutates state or communicates externally. A clear request to perform a routine send, create, update, clipboard read, or clipboard write is sufficient authorization when all required details are present. Drafting an email is not sending it and does not require confirmation.

Unsafe actions, impersonation, credential disclosure, bulk unsolicited messaging, and unavailable destructive operations must be refused rather than confirmed.

Scenario Files

Scenario fixtures live under `scenarios/v1/`.

The established combined benchmark is `scenarios/v1/` plus `scenarios/v2/` (150 scenarios). The separate `scenarios/template_shift_v1/` suite is a locked 30-scenario diagnostic for testing ACT, CLARIFY, and REFUSE behavior under new wording and hard keyword counterexamples. It must not be used to construct training examples, and its results must be reported separately from the established benchmark.

Each scenario must validate against:

```
src/baymax/eval/schemas/scenario.schema.json
```

Each scenario includes `current_time` so relative phrases like "tomorrow", "Friday", or "in two hours" resolve deterministically during eval runs.

Each scenario defines:

- the user input
- the current time
- the available tools
- the initial fake tool state
- the expected behaviour
- the success criteria

Phase 1 Metrics

The Phase 1 runner will report the following metrics:

Task Success

Whether the scenario reached the expected final outcome.

Tool-Call Accuracy

Whether the agent selected the correct tool for the scenario.

Argument Accuracy

Whether the agent supplied correct tool arguments, such as date, time, title, recipient, or duration.

Clarification Accuracy

For ambiguous scenarios, whether the agent asked for the required missing information instead of guessing or taking premature action.

Hallucination Rate

How often the agent invents unsupported actions, calls unavailable tools, claims success without tool execution, or fabricates state not present in the scenario.

Latency

The elapsed time for the scenario run, reported at minimum as total latency per scenario. Later runs may include P50 and P99 latency.

Cost

The estimated model/API cost per scenario and per benchmark run.

Phase 1 Output

The first scripted baseline output target is:

```
results/phase1/scripted/v1-baseline.json
```

The first OpenAI agent baseline output target is:

```
results/phase1/openai/gpt-4o-mini-full.json
```

This file should include:

- run ID
- model/backend name
- scenario count
- aggregate metrics
- per-scenario pass/fail results
- failure reasons where applicable
- timestamp
- benchmark configuration

Phase 1 Commands

Run the scripted baseline:

```
python -m baymax.eval.cli run-scripted `
  --scenarios scenarios\v1 `
  --responses scenarios\v1\scripted_responses\v1-scripted.json `
  --output results\phase1\scripted\v1-baseline.json
```

Run the OpenAI agent baseline:

```
$env:OPENAI_API_KEY="your_api_key"
python -m baymax.eval.cli run-agent-openai `
  --scenarios scenarios\v1 `
```

```
  --output results\phase1\openai\gpt-4o-mini-full.json `
  --model gpt-4o-mini
```

Run local verification:

```
uv run ruff format --check
uv run ruff check .
uv run pyright
uv run pytest
```

Phase 1 Baseline Results

The scripted baseline is expected to pass all 50 scenarios because it uses hand-authored responses that match the scenario fixtures.

The first OpenAI agent baseline was run against the same 50 scenarios:

```
backend: openai:gpt-4o-mini
scenario_count: 50
task_success_rate: 0.52
average_tool_call_accuracy: 0.82
average_argument_accuracy: 0.9083333333333333
average_clarification_accuracy: 0.5555555555555556
average_refusal_accuracy: 0.21428571428571427
average_hallucination_rate: 0.0
average_latency_ms: 2518.9
total_cost_usd: 0.0058130000000000005
```

The main baseline weakness is boundary judgment: the general agent often takes action when the expected behavior is to clarify, refuse, or wait for confirmation. That result is useful because BAYMAX's target local agent should be trained and evaluated against these domain-specific rules, not only generic tool-use ability.

Phase 2 Decision-Boundary Ablation

The following Qwen 2.5 1.5B runs used the same runtime prompt, LoRA settings, seed, one epoch, 1,000-row training cap, and evaluation runner. The controlled data changes altered which unique BAYMAX-control examples survived the cap. The established benchmark combines `scenarios/v1/` and `scenarios/v2/`.

Run	Success	Tool	Argument	Clarify	Refuse	Hallucination
Base Qwen	0.213	0.493	0.791	0.025	0.034	0.027
ACR-1k	0.287	0.567	0.761	0.050	0.172	0.020
Schema-diverse	0.280	0.560	0.752	0.075	0.190	0.007
ACT-balanced	0.313	0.553	0.805	0.025	0.086	0.010

ACT-balanced increased the fixed BAYMAX contribution from 300 to 400 unique rows: 200 ACT, 100 CLARIFY, and 100 REFUSE. This reduced missing tool calls from 34 to 22 relative to schema-diverse, but increased premature tool calls from 32 to 44. It produced the best established-suite task success and argument accuracy, while schema-diverse retained better clarification and refusal accuracy. Neither checkpoint dominates the other.

The locked 30-scenario template-shift diagnostic showed the same tradeoff:

Run	Success	Tool	Argument	Clarify	Refuse	Hallucination
Base Qwen	0.233	0.483	0.947	0.000	0.000	0.033
ACR-1k	0.200	0.600	0.800	0.200	0.150	0.000
Schema-diverse	0.167	0.533	0.733	0.300	0.200	0.000
ACT-balanced	0.200	0.533	0.828	0.200	0.100	0.000

Strict template-shift passes changed from 2/10 ACT and 3/10 CLARIFY for schema-diverse to 4/10 ACT and 2/10 CLARIFY for ACT-balanced. Both had 0/10 strict REFUSE passes. This indicates that changing the class mix shifts the model's action prior; it does not by itself teach a robust semantic decision boundary.

Refusal Audit

On the ten template-shift refusal scenarios, schema-diverse emitted three `refuse_request` calls, four `request_clarification` calls, two productive-tool responses, and one empty response. ACT-balanced emitted two refusals, three clarifications, four productive-tool responses, and one empty response. Every actual refusal matched only one of its scenario's two required reason keywords, so it received `0.5` refusal accuracy and did not count as task success.

These result artifacts predate explicit decision tracking. At that time, the local runner converted both meta-tools into a message and the scorer used only keyword overlap. New runs preserve `act`, `clarify`, or `refuse` as `AgentResponse.decision_type` and report decision accuracy separately from question/reason wording. Historical artifacts remain unchanged and report no decision-accuracy value.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.